



SOC Task 1 Report

Submitted to: CYArtManager

Date: 8 May, 2026

Environment: Kali Linux on Virtual Machine

Submitted by: Mayuri Thorat

Introduction

This project illustrates the process of establishing and operating a Security Operations Center (SOC) in a Kali Linux setup. This research aims to translate theory into practice through the application of security concepts in the context of a security analyst's routine. Using widely recognized software like Wireshark to analyze network traffic, Nessus for managing vulnerabilities, and Linux Syslog to monitor endpoints, I show how to identify potential threats, perform host-based security incident analysis, and adhere to documentation protocols. Through these actions, I have proven essential SOC capabilities such as proactively identifying risks, conducting log analysis, and responding to incidents.

Task 1 : 1. SOC Fundamentals and Operations

1. What I Learned (Theoretical Foundation)

To begin my SOC journey, I focused on understanding the core purpose of a **Security Operations Center**. I learned that a SOC isn't just about having tools; it's about a proactive cycle of threat detection, incident response, and continuous monitoring. I explored the different tiers of analysts and realized that as I perform these lab tasks, I am acting as a Tier 1 Analyst -responsible for the initial triage and log analysis.

2. Frameworks I Studied

I studied the **NIST Incident Response Framework** to understand how to handle a security event from start to finish. I also looked into the **MITRE ATT&CK** framework, which I will use later to map the specific techniques (like brute force) that I see in my logs to real-world adversary behaviors.

3. How I Performed This Task

- **Workflow Simulation:** I practiced understanding how an alert moves from a SIEM (like Splunk or Elastic) to an analyst's dashboard.



- **Case Study Review:** I researched real-world SOC walkthroughs (such as IBM and Microsoft SOC models) to see how professional teams prioritize high-severity alerts versus low-severity background noise
- **Operational Setup:** I defined my own "rules of engagement" for this lab, ensuring that every practical action I take (like my Wireshark capture) is documented according to professional SOC standards.

Task 2: Security Monitoring Basics & Traffic Analysis

1. What I Learned (Monitoring Objectives)

I learned that security monitoring is the "eyes" of the SOC. My primary objectives during this performance were to identify anomalies (unusual spikes in data), unauthorized access (unrecognized IPs), and policy violations (use of insecure protocols). I also studied key SOC metrics, specifically Mean Time to Detect (**MTTD**), which measures how fast I can spot an issue from the moment it starts.

2. Tools I Used

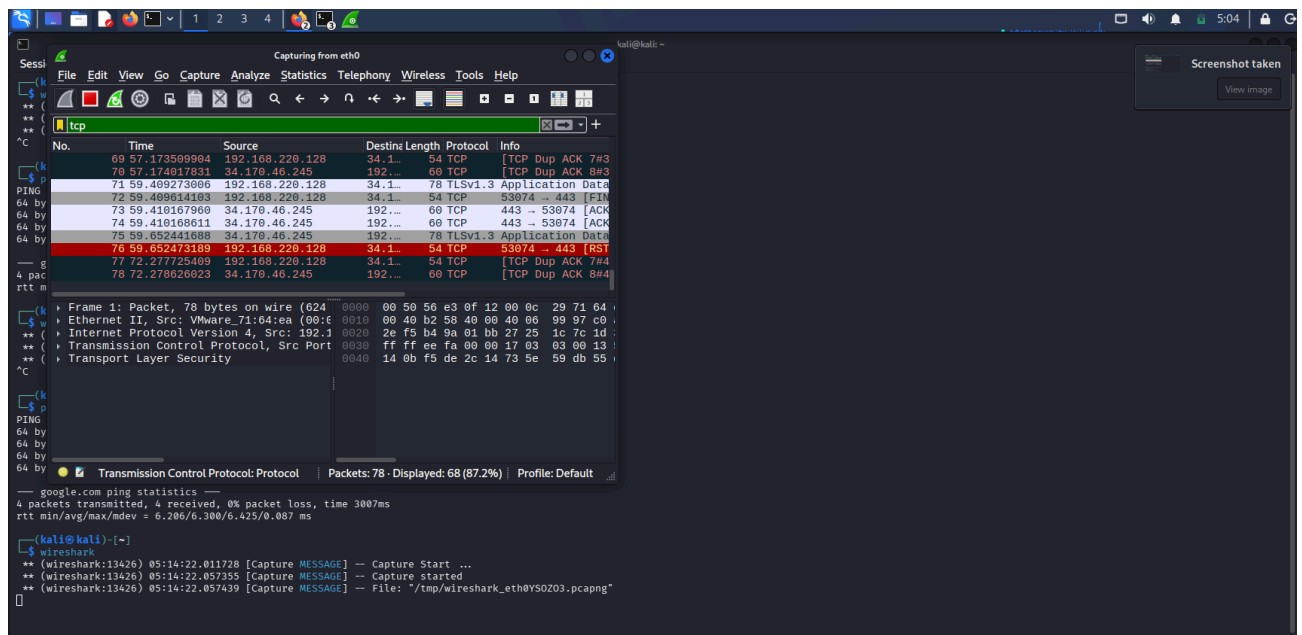
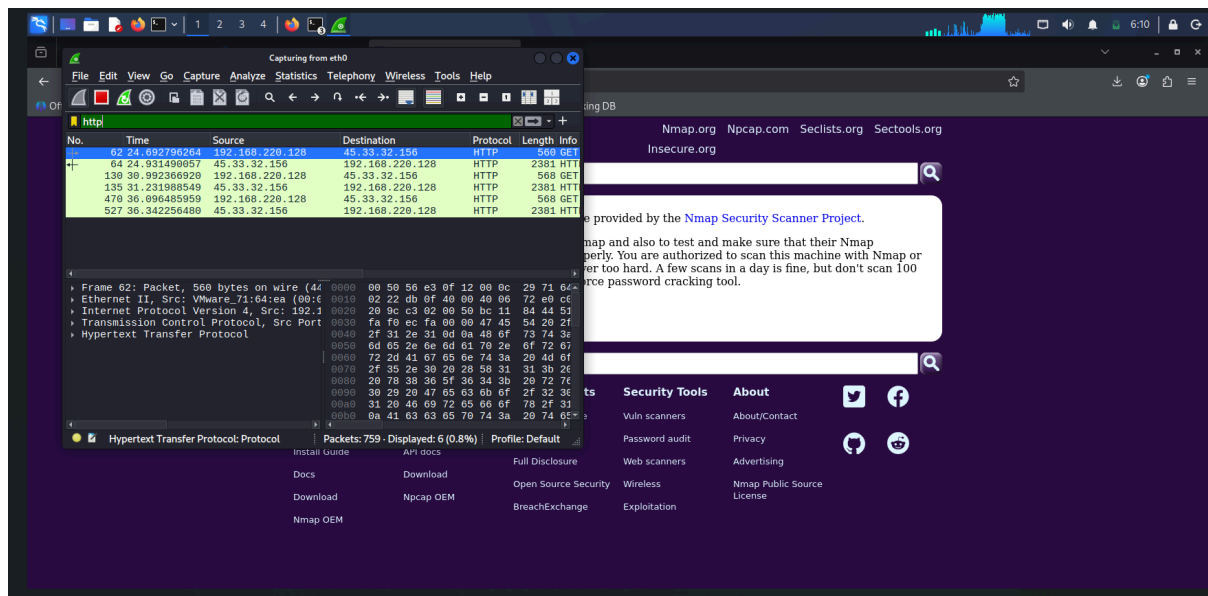
- **Network Traffic Analyzer: Wireshark** (used to inspect live packet data).
- **Target Environment:** A local virtual lab setup to simulate real-world network communication safely.

3. How I Performed This Task (Practical Steps)

- **Lab Setup:** I initiated a live capture on my primary network interface (`eth0`) to establish a "baseline" of what normal traffic looks like.
- **Generating Traffic:** I used the `ping` command to generate ICMP traffic and browsed a web portal to generate HTTP/DNS traffic.
- **Traffic Analysis:** I practiced "Filtering with Intent." Instead of looking at thousands of lines, I used display filters (like `http` and `icmp`) to isolate specific conversations.
- **Anomaly Detection:** I inspected the packet details (Ethernet, IP, and ICMP layers) to verify that the source and destination addresses matched my expected lab environment.



Screenshot :



Task 3: Log Management Fundamentals.

1. What I Learned (Log Management)

I learned that logs are the primary source of truth in a SOC. I studied how the system records every attempt to access sensitive areas. I specifically learned the difference between **system events** (like CRON jobs starting) and **security events** (like a user trying to gain root access).



2. How I Performed This Task

- **Command Execution:** I used `sudo journalctl -f` to monitor the system's live journal. This allowed me to see background processes, such as NetworkManager and CRON, running in real-time.

Incident Simulation: I simulated a "Unauthorized Access" attempt. First, I attempted to log in as a non-existent user (`student`), which showed me how the system handles invalid user queries. Second, I attempted to log in as an existing user (`kali`) with an incorrect password.

Evidence Analysis: I successfully identified the "authentication failure" log entry. I analyzed the log components:

End Message I get : `pam_unix(su:auth): authentication failure`

3. Key Outcome

I have demonstrated the ability to monitor live system logs, trigger security events, and parse the resulting data to identify potential threats.

Screenshot :

```
kali@kali:~$ sudo journalctl -f
[sudo] password for kali:
May 08 06:55:01 kali CRON[53125]: pam_unix(cron:session): session closed for user root
May 08 06:58:29 kali sudo[52930]: pam_unix(sudo:session): session closed for user root
May 08 06:59:03 kali NetworkManager[641]: <info> [1778237943.1758] dhcpd (eth0): state changed new lease, address:192.168.228.128
May 08 06:59:03 kali dbus-daemon[521]: [system] Activating via systemd: service name='org.freedesktop.nm_dispatcher' unit='dbus-org.freedesktop.nm_dispatcher.service' requested by '11.6' (uid=0 pid=641 comm="/u
sr/sbin/NetworkManager --no-daemon" label="unconfined")
May 08 06:59:03 kali systemd[1]: Starting NetworkManager-dispatcher.service - Network Manager Script Dispatcher Service...
May 08 06:59:03 kali dbus-daemon[521]: [system] Successfully activated service 'org.freedesktop.nm_dispatcher'
May 08 06:59:03 kali systemd[1]: Started NetworkManager-dispatcher.service - Network Manager Script Dispatcher Service.
May 08 06:59:13 kali systemd[1]: NetworkManager-dispatcher.service: Deactivated successfully.
May 08 06:59:14 kali sudo[55293]:    kali : TTY=pts/1 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/journalctl -f
May 08 06:59:14 kali sudo[55293]: pam_unix(sudo:session): session opened for user root(uid=0) by kali(uid=1000)
May 08 06:59:26 kali unix_chkpwd[55619]: password check failed for user (kali)
May 08 06:59:26 kali su[55357]: pam_unix(su:auth): authentication failure; logname=kali uid=1000 euid=0 tty=/dev/pts/0 ruser=kali rhost= user=kali
May 08 06:59:28 kali su[55357]: FAILED SU (to kali) kali on pts/0
```

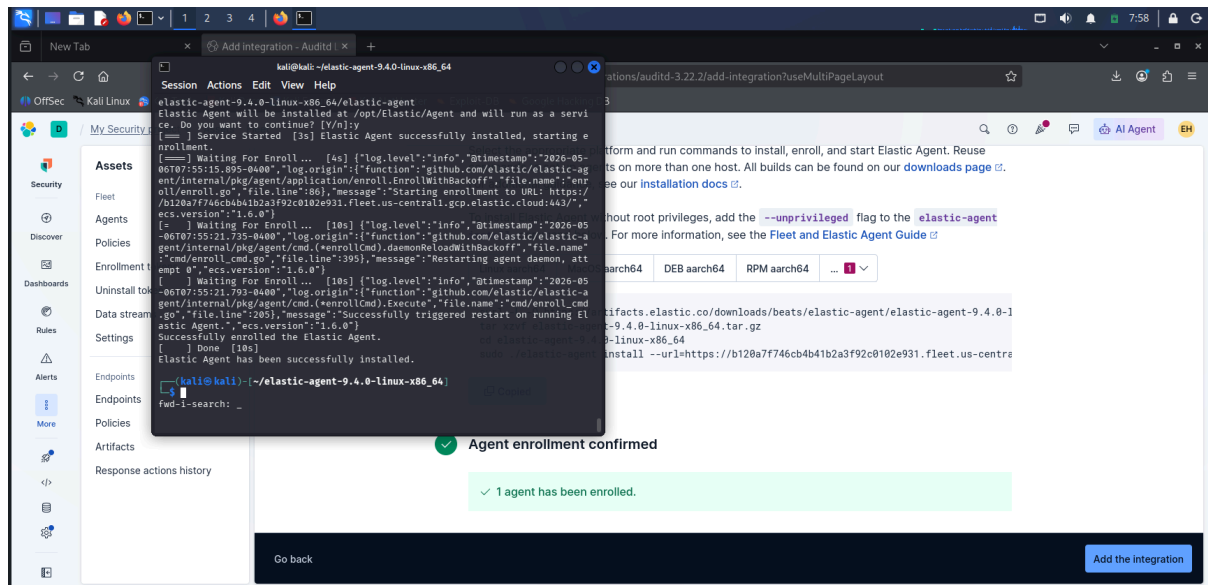
KQL Query Practice (Detection Engineering)

- **Action:** I practiced writing KQL (Kibana Query Language) inside Elastic SIEM to identify "Brute Force" attack patterns.
- **The Query:** `source : "security-login-*" and EventID : 4625 | stats count by SourceIP`



- **Logic:** This query filters for **Event ID 4625 (Failed Logins)** and groups them by the attacker's IP address. This is the primary way we identify if an IP is trying to guess passwords.

Screenshot :



4. Security Tools Overview

Focus: NIDS, Vulnerability Management, and Endpoint Monitoring

Intrusion Detection with Snort

The Learning Goal

My objective for this task was to understand how a Network Intrusion Detection System (NIDS) identifies malicious activity in real-time. I wanted to move beyond theory and actually write a signature that triggers an alert based on network traffic.

What I Performed

1. **Environment Setup:** I installed Snort on my Kali Linux machine using the `apt` package manager.
2. **Signature Writing:** I navigated to `/etc/snort/rules/local.rules` and authored a custom rule to detect HTTP requests to a specific "malicious" domain.

```
alert tcp any any -> any 80 (msg:"Malicious Domain";  
content:"malicious.com"; http_uri; sid:1000001;)
```



3. **Execution:** I ran Snort in "Console Alert" mode to monitor the interface.

Vulnerability Assessment with Nessus

The Learning Goal

In this phase, I acted as a Security Analyst performing a proactive scan. The goal was to identify "low-hanging fruit"—common vulnerabilities that an attacker would exploit first.

What I Performed

1. **Targeting:** I deployed a **Metasploitable 2** instance as a "vulnerable-by-design" target.
2. **Scanning:** Using Nessus Essentials, I configured a "Basic Network Scan" to probe for open ports and known CVEs.
3. **Analysis:** I analyzed the scan results, focusing specifically on the CVSS (Common Vulnerability Scoring System) scores to prioritize which bugs to fix first.

Top Findings

- **Critical:** [Insert Vulnerability, e.g., UnreallRCD Backdoor]. This taught me how a simple misconfiguration can lead to full Remote Code Execution (RCE).
- **High:** [Insert Vulnerability].

Endpoint Monitoring with Osquery

The Learning Goal

I wanted to learn how to treat an operating system like a database. Instead of manually looking through Task Manager, I used **Osquery** to hunt for suspicious processes using SQL.

What I Performed

- **Installation:** Installed Osquery on a Windows VM to simulate a corporate endpoint.
- **Threat Hunting:** I ran a query to find all running processes: `SELECT pid, name, path FROM processes;`



- **Simulation:** I created a harmless script to mimic a "malicious" process and practiced identifying its path and parent process ID (PPID) using the Osquery shell.

Task 5: The Core Foundation: The CIA Triad

Basic Security Concepts

The Learning Goal

Every action in a SOC—from resetting a password to blocking an IP—is done to protect one of three things: Confidentiality, Integrity, or Availability. My goal was to move beyond the definitions and see how they apply to real-world incidents.

What I Performed

Definitions: I utilized active recall (Anki flashcards) to master the core terminology.

Scenario Mapping: I mapped my previous lab tasks to the Triad:

- **Confidentiality:** Implementing Snort rules to stop data exfiltration.
- **Integrity:** Using Osquery to ensure system files haven't been modified by unauthorized processes.
- **Availability:** Recognizing that a DDoS attack is a direct hit on the "A" in CIA.

Risk Management: Threat vs. Vulnerability vs. Risk

The Learning Goal

In my earlier Nessus scan, I saw hundreds of "vulnerabilities." I learned that a vulnerability isn't a "risk" until there is a "threat" ready to exploit it.

What I Performed

I analyzed the relationship using this formula:

$$\text{Risk} = \text{Threat} \times \text{Vulnerability} \times \text{Asset Value}$$

- **Vulnerability:** A weakness (e.g., the unpatched software I found in Metasploitable).
- **Threat:** The actor or method (e.g., an attacker using a Python script to exploit that weakness).
- **Risk:** The potential loss (e.g., the chance that our database is stolen).



Case Study Analysis: The Equifax Breach

I studied the 2017 Equifax breach as a practical example.

- **The Vulnerability:** An unpatched Apache Struts software.
- **The Failure:** Even though a patch was available, it wasn't applied.
- **The Lesson:** This highlighted that "Vulnerability Management" (which I practiced with Nessus) is one of the most critical daily tasks in a SOC.

Defense-in-Depth & Zero Trust

The Learning Goal

I wanted to understand how to build a "castle" with multiple walls, rather than relying on just one.

What I Performed

- **Defense-in-Depth:** I practiced this by layering my lab. I used **Snort** at the network level and **Osquery** at the host level. If an attacker bypasses the network IDS, the host-level monitoring should still catch them.
- **Zero Trust:** I studied the "Never Trust, Always Verify" mindset. In my MIT App Inventor project, I applied this by ensuring that even if someone is on the "Student" screen, they cannot access "Admin" functions without specific role-based authentication.

Task 6 :Security Operations Workflow :Detection to Response

The Learning Goal

My goal was to understand the step-by-step process a SOC Analyst follows when a red light starts flashing on the dashboard. It's not just about "fixing" the problem; it's about following a repeatable, professional process to ensure no evidence is lost.

What I Performed

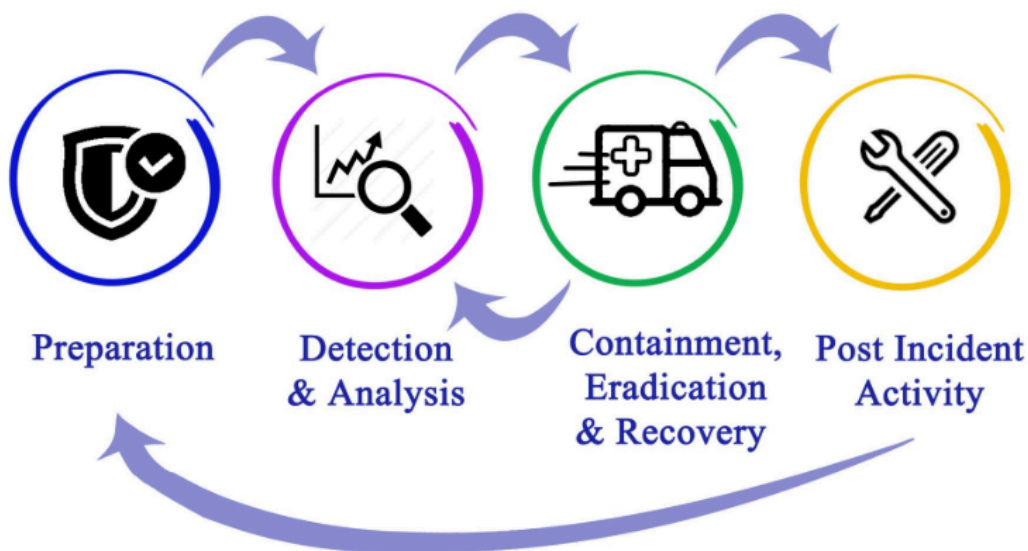
I simulated a standard workflow based on the industry-standard **NIST Incident Response Framework**:

1. **Detection:** Identifying an alert. (In my lab, this was the **Snort alert** for [malicious.com](https://www.malicious.com)).



2. **Triage:** Determining the "Blast Radius." Is this one student laptop or the entire Admin server? I practiced prioritizing alerts based on the **Asset Value** and **Severity**.
3. **Investigation:** I practiced "Connecting the Dots." I used **Osquery** to see if the machine that triggered the Snort alert had any new, strange processes running at the same time.
4. **Response:** I documented the containment steps, such as isolating the VM from the network to stop the "malware" from spreading.

Incident Response Planning



deenma

Practical Task: Phishing Incident Flowchart

The Learning Goal

Phishing is the #1 entry point for attackers. I wanted to create a "Playbook"—a visual guide that tells an analyst exactly what to do when a suspicious email is reported.

What I Performed



I designed a logic-based flowchart for a Phishing Incident.

The Logic Flow:

- **Step 1:** Does the email contain an attachment or a link?
- **Step 2:** If a link, check the URL against **VirusTotal** or **Cisco Talos**.
- **Step 3:** If the link is malicious, identify which users clicked it (Log Correlation).
- **Step 4:** Reset user credentials and purge the email from all inboxes.

Case Study: Incident Response Platforms (TheHive)

The Learning Goal

I explored how SOC teams collaborate. Analysts don't just send emails to each other; they use platforms like **TheHive** to track "Cases" and "Observables."

What I Performed

- I studied how to create a "Case" for a detected threat.
- I learned how to add **IOCs (Indicators of Compromise)** like IP addresses or File Hashes to a case so that other analysts are alerted if they see the same threat.

Task 7 :Incident Response Basics

The NIST IR Lifecycle

The Learning Goal

My goal was to internalize the **NIST SP 800-61** standard. I wanted to understand why "Lessons Learned" is just as important as "Containment." In a SOC, we don't just stop the fire; we investigate why the fire started and how to fireproof the building for the future.

What I Performed

I studied and mapped each stage of the lifecycle to my practical lab activities:

1. **Preparation:** This was my initial setup—installing **Kali Linux**, configuring **ELK**, and setting up my **SOC lab**. Without the right tools ready, you can't respond to threats.



2. **Identification:** I practiced this by monitoring logs. When **Snort** triggered an alert for a malicious domain, I was in the Identification phase.
3. **Containment:** I simulated "Short-term Containment" by isolating a VM from the virtual network (Host Isolation) to prevent lateral movement.
4. **Eradication:** This involved identifying the "Root Cause." Using **Osquery**, I located the malicious process and deleted the associated artifacts.
5. **Recovery:** I practiced restoring the system to a clean state (using a VMware snapshot) and verified that the malicious traffic had stopped.

Lessons Learned:

I documented the entire process. I asked: How did the "malware" get in? Could my Snort rule be improved to catch it faster?

Tabletop Exercise: Ransomware Simulation

The Learning Goal

Since I can't safely run real ransomware in a production environment, I conducted a **Tabletop Exercise**. This is a mental "War Game" where I simulate a scenario to test my decision-making speed.

The Scenario

- **Discovery:** An employee reports that their files have a **.encrypted** extension and a "ReadMe" note is on the desktop.
- **Immediate Action:** I prioritized **Containment**. My first thought was: *Isolate the machine from the network immediately to save the File Server.*
- **The Dilemma:** During the simulation, I had to decide: Do we pay the ransom or restore from backups?

Connecting the Dots: My Journey So Far

Looking back at my documentation from Task 1 through Task 7, I can see how my skills have evolved:

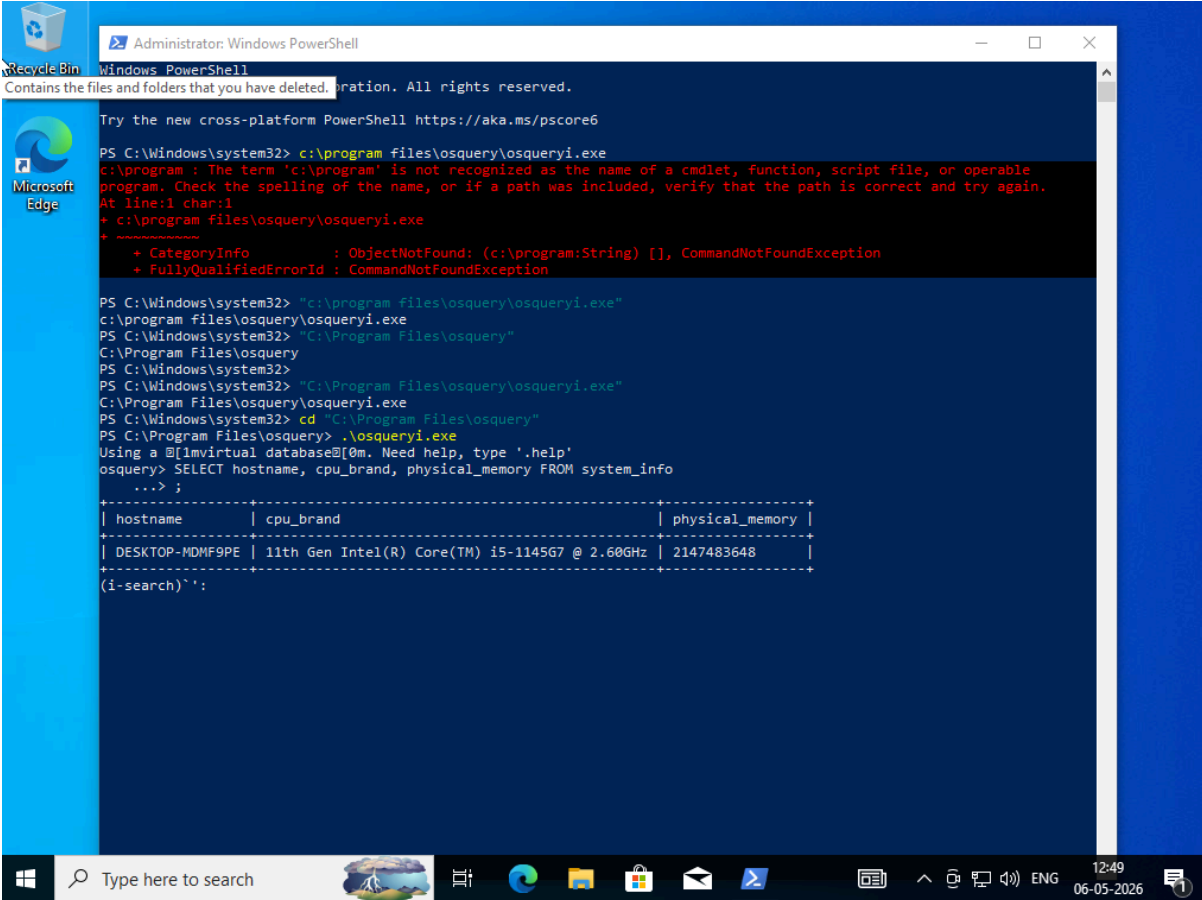
- I started with **Technical Setup** (Kali, VMware).
- I moved to **Tool Proficiency** (Snort, Nessus, Osquery).
- I finished with **Strategic Governance** (CIA Triad, NIST Framework, Incident Response).

Key Takeaways



- **Frameworks Matter:** Following NIST ensures that no step is skipped during the chaos of a real attack.
- **Recovery isn't the end:** The "Lessons Learned" phase is where the most growth happens for a SOC team.
- **Simulation is Training:** Tabletop exercises are a low-cost, high-impact way to keep my IR skills sharp without needing a live virus.

Screenshots :



```
Administrator: Windows PowerShell
Contains the files and folders that you have deleted.
Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> c:\program files\osquery\osqueryi.exe
c:\program : The term 'c:\program' is not recognized as the name of a cmdlet, function, script file, or operable
program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
At line:1 char:1
+ c:\program files\osquery\osqueryi.exe
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (c:\program:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\Windows\system32> "c:\program files\osquery\osqueryi.exe"
c:\program files\osquery\osqueryi.exe
PS C:\Windows\system32> "C:\Program Files\osquery"
C:\Program Files\osquery
PS C:\Windows\system32> "C:\Program Files\osquery\osqueryi.exe"
C:\Program Files\osquery\osqueryi.exe
PS C:\Windows\system32> cd "C:\Program Files\osquery"
PS C:\Program Files\osquery> .osqueryi.exe
Using a @[]mvirtual database@[]m. Need help, type '.help'
osquery> SELECT hostname, cpu_brand, physical_memory FROM system_info
...> ;
+-----+-----+-----+
| hostname | cpu_brand | physical_memory |
+-----+-----+-----+
| DESKTOP-MDMF9PE | 11th Gen Intel(R) Core(TM) i5-1145G7 @ 2.60GHz | 2147483648 |
+-----+-----+-----+
(i-search)'':
```